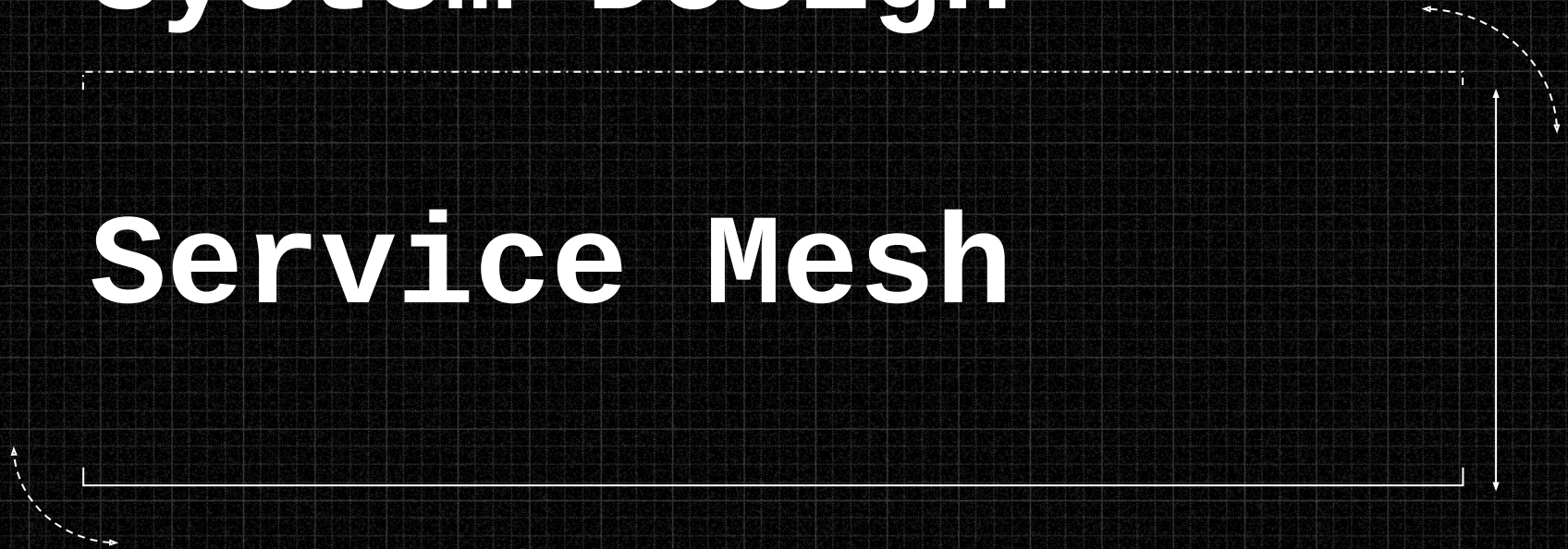


System Design

Service Mesh



\$ whoami

Matheus Fidelis

Engenheiro de \$RANDOM

@fidelissauro

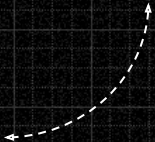
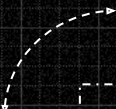
<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>



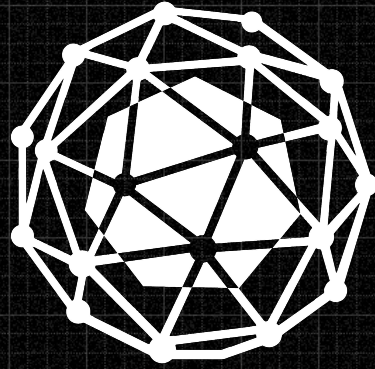
OBJETIVOS

- ✓ CONCEITOS DE NETWORKING
- ✓ FUNDAMENTOS E PROPOSTAS
- ✓ COMPARATIVOS DE MERCADO



1

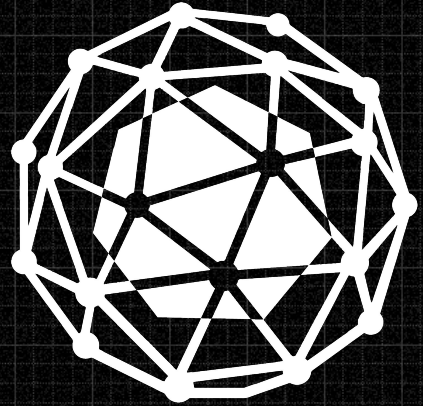
SERVICE MESH



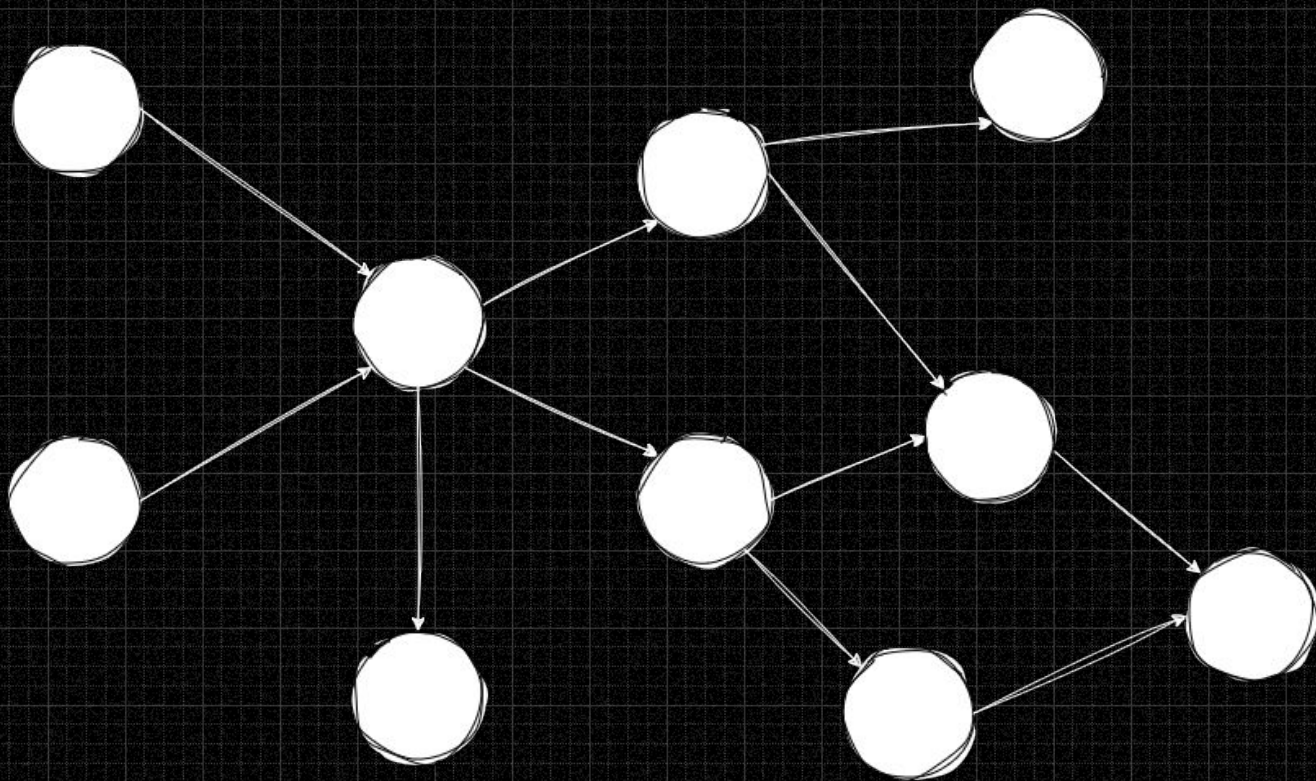
TRATAR NETWORKING COMO SOFTWARE

Service Mesh

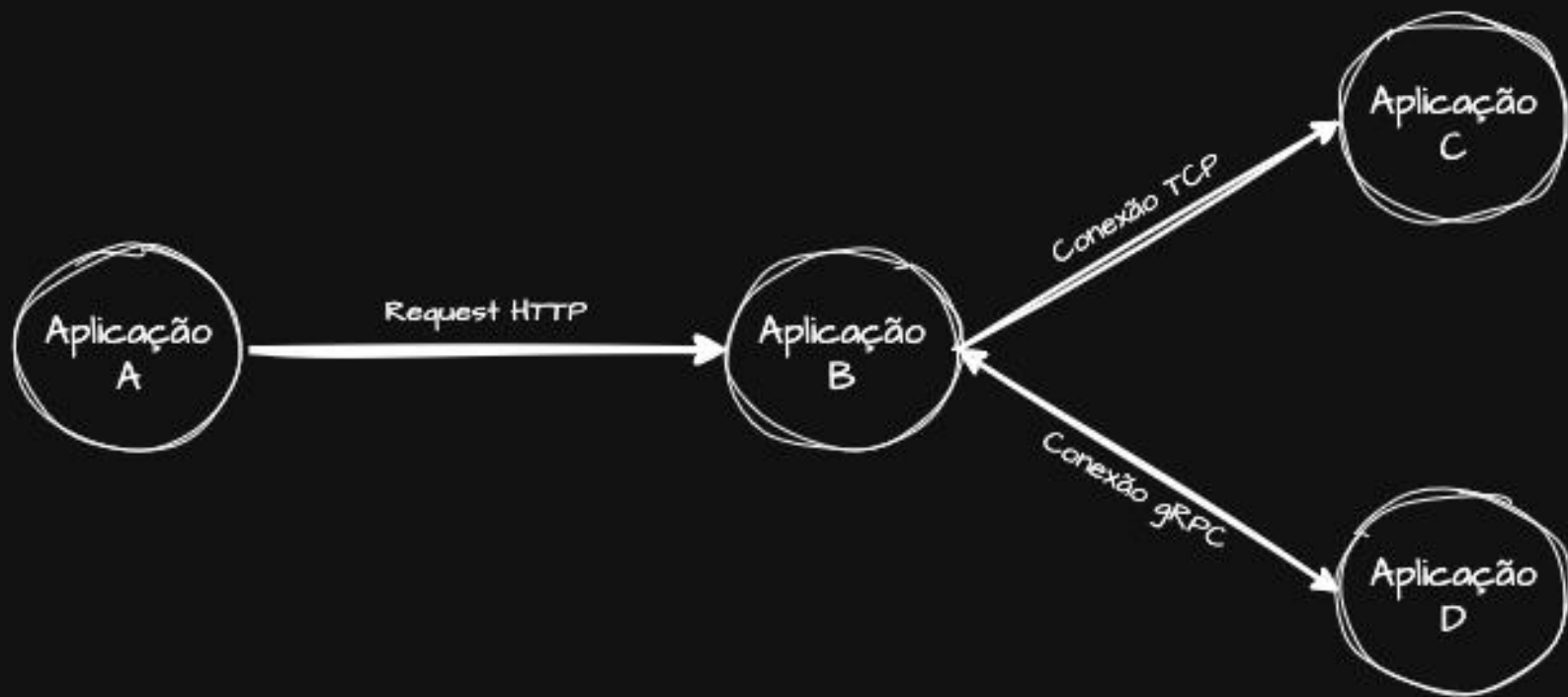
- Pattern de Networking
- Complexidade da comunicação
- Ambientes de Microserviços
- Comunicação Síncrona
- Mapear todos os Pontos de Contato
- Mapeamento e Governança
- Monitorar granular
- Adicionar comportamentos
- Extender capacidades de comunicação



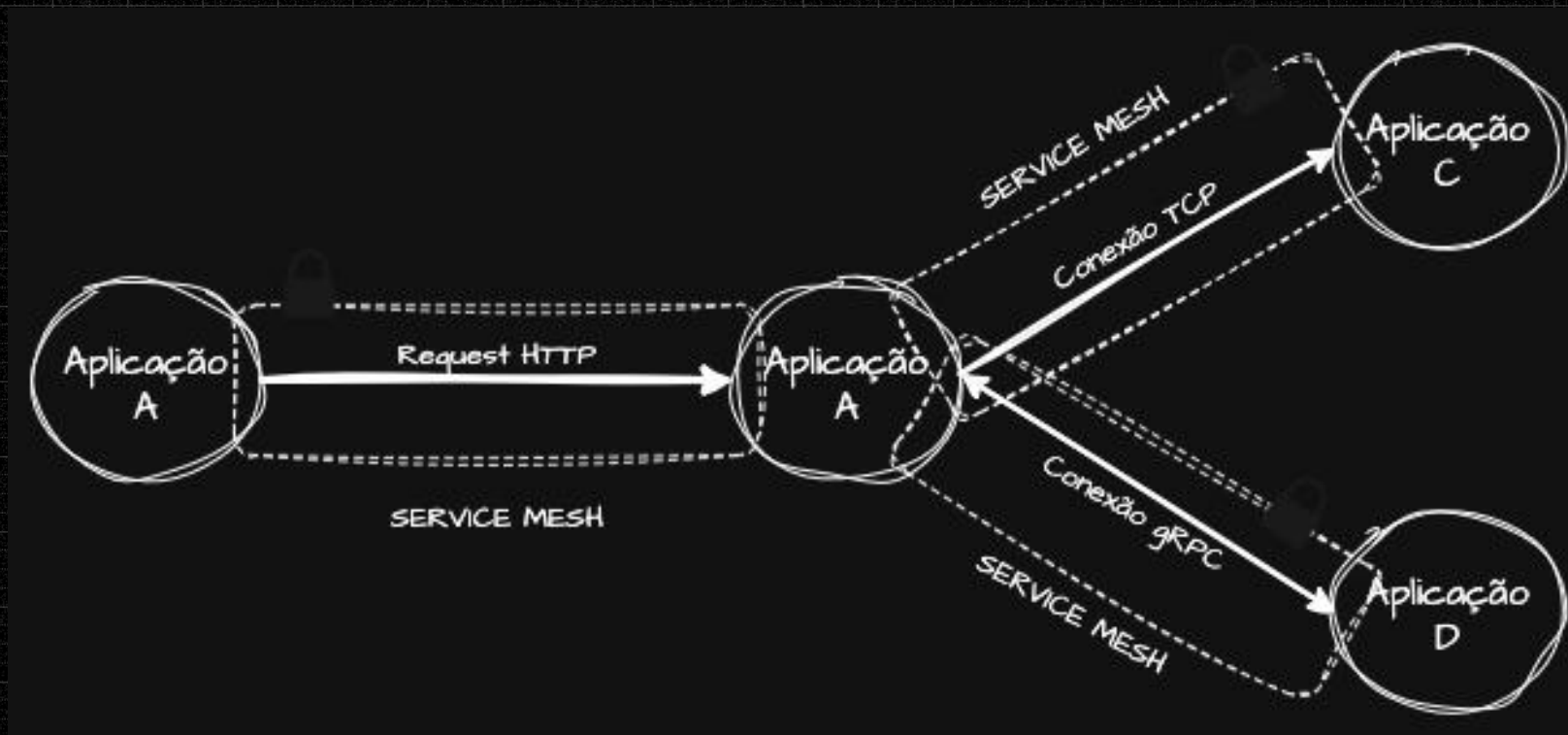
SERVICE MESH

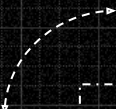


SERVICE MESH



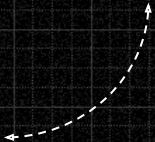
SERVICE MESH





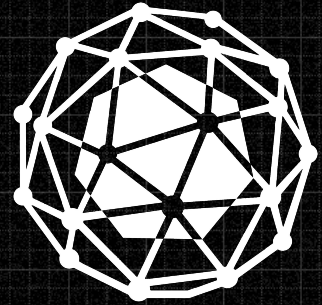
2

Features de Service Mesh



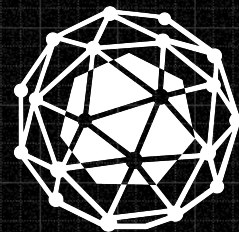
Features

- Interceptação
- Roteamento Avançado
- Modificação dos Requests
- Retries na camada de comunicação
- Controle de Recursos
 - Circuit Breakers, Connection Pool
 - Keep Alive, Rate Limit, Health Check
- Segurança
 - Criptografia (mTLS)
 - Autenticação e Autorização
- Metrificação

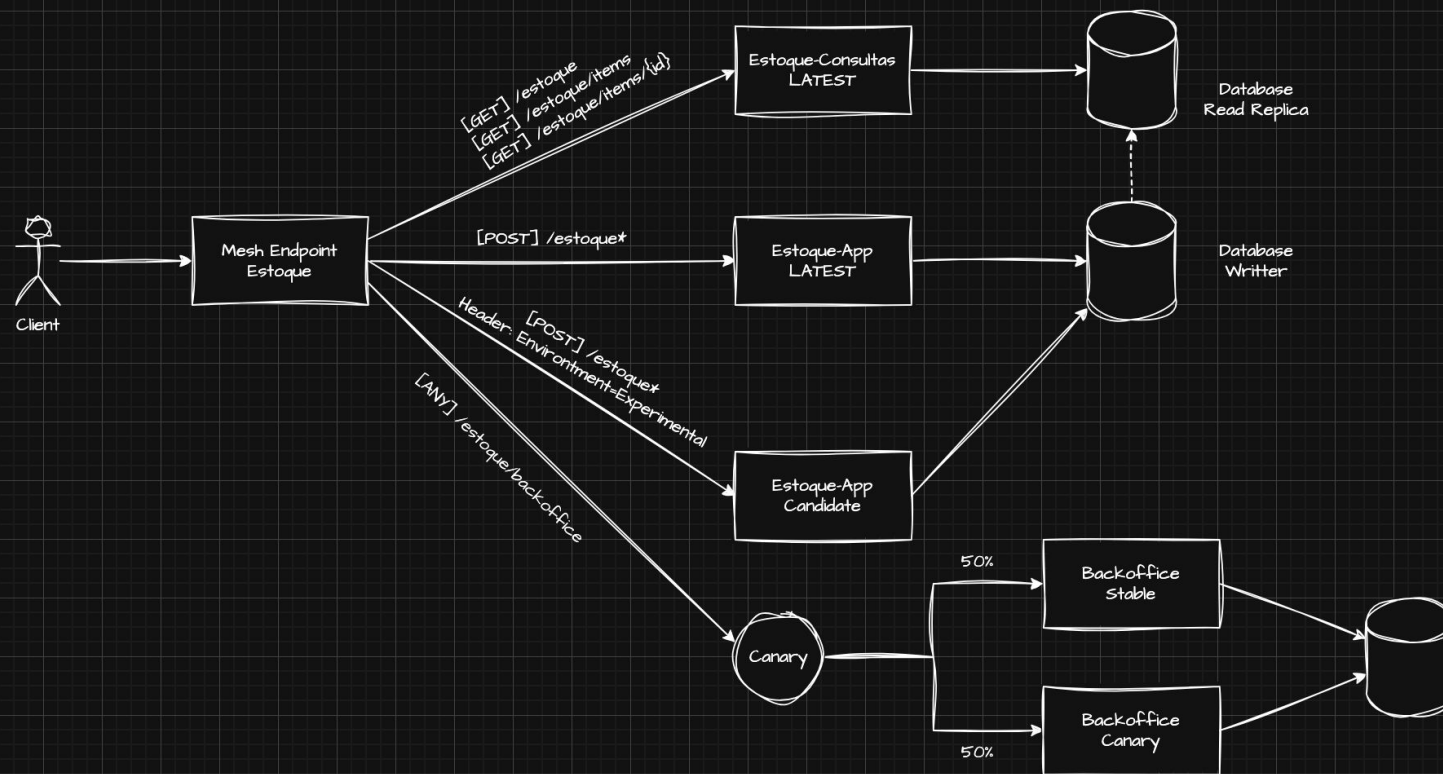


Roteamento Avançado

- Roteamento de forma granular
- Definir regras sofisticadas de roteamento de tráfego entre serviços
- Encaminhar requisições:
 - Cabeçalhos, paths, versões ou pesos de tráfego
- Modelos de Deployment
- Totalmente transparente da aplicação

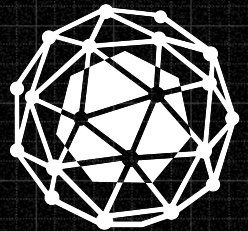


Roteamento Avançado

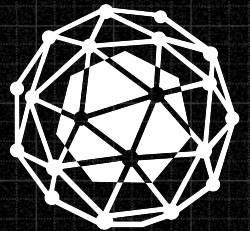
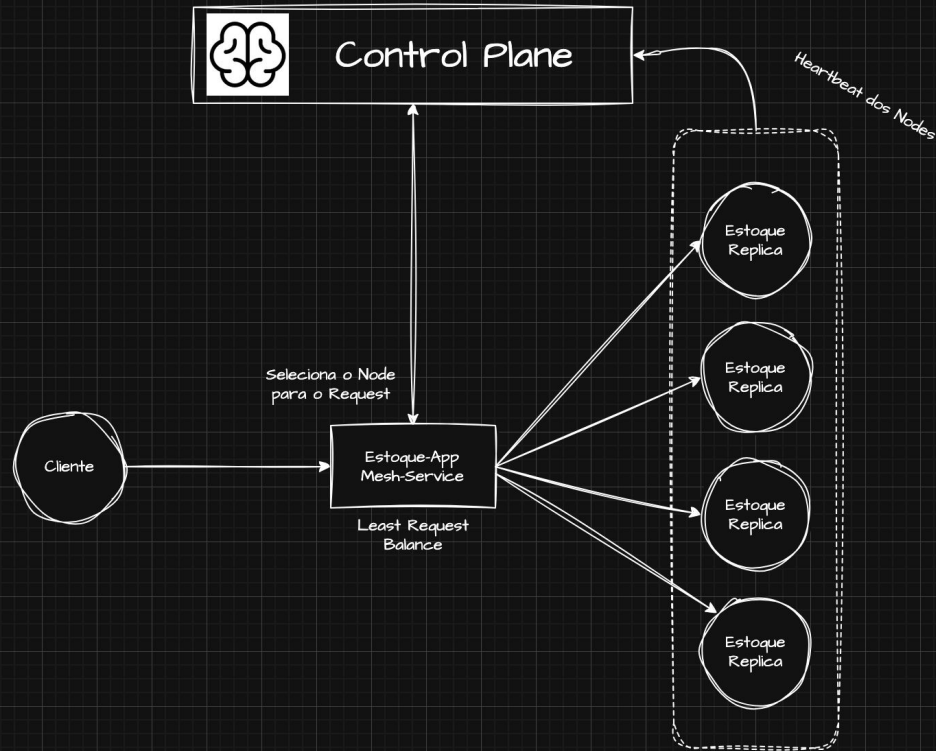


Balanceamento de Carga

- Balanceamento Complexo
- Diversos algoritmos
- Interceptação de Request e Response
- Warm Up de Novos Hosts
- Linear
- Retries e Circuit Breakers
- Healthchecks

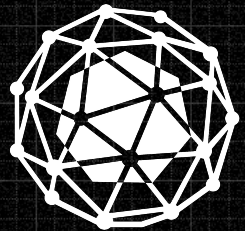


Balanceamento de Carga

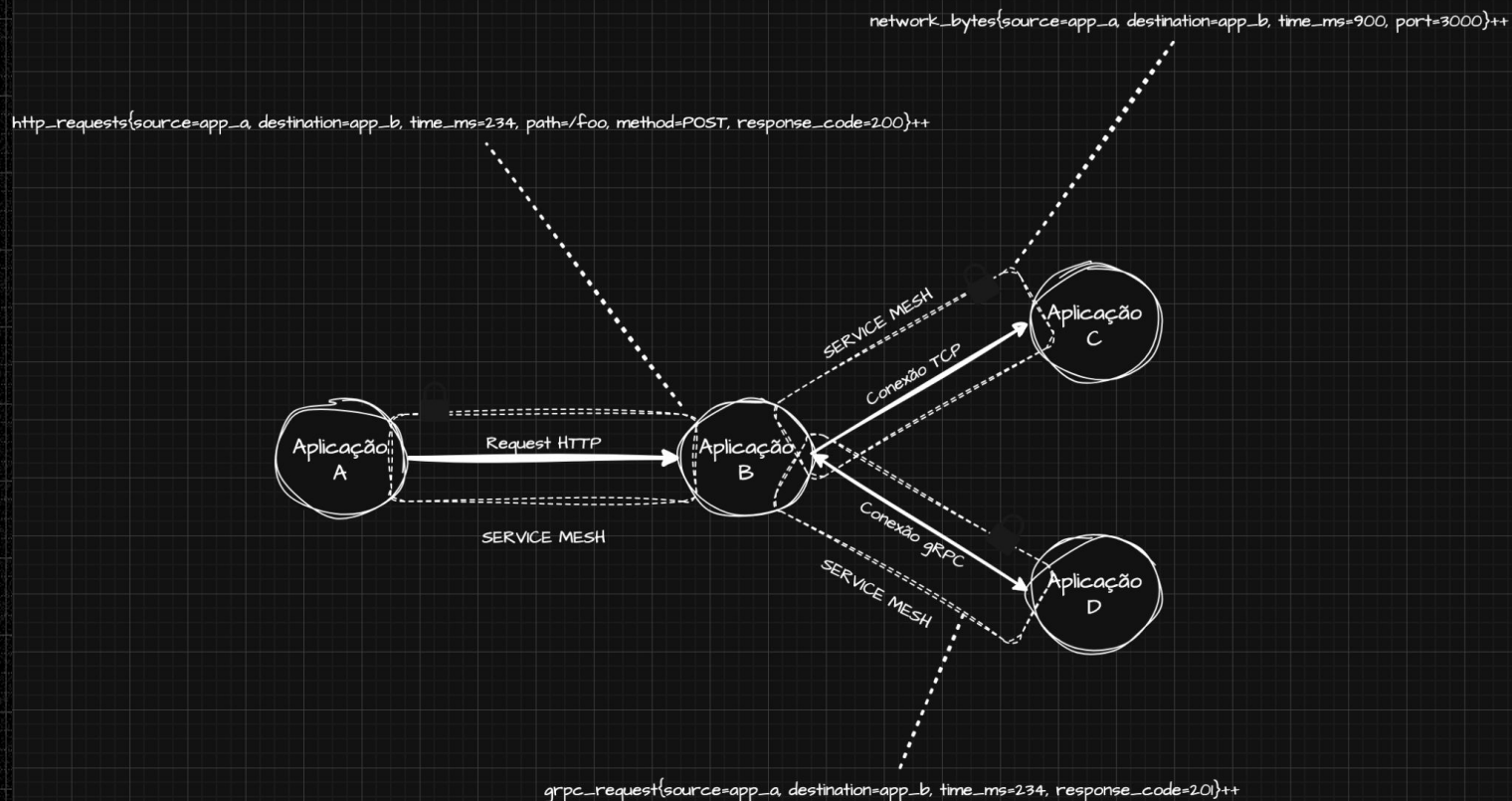


Observabilidade Desacoplada

- Observabilidade de dia zero
- Interceptar e adicionar comportamentos customizados diretamente nas conexões e requisições
- Incluir métricas de latência, taxa de erro, throughput e tempo de resposta
- Gerar spans de tracing distribuído

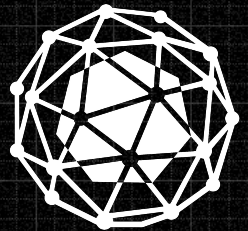


Observabilidade Desacoplada



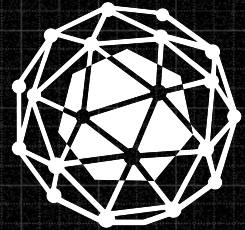
Segurança, Autenticação e Autorização

- Segurança e isolamento de forma altamente performática e transparente
- Identificar quais são os membros de determinados grupos de microserviços
- Controles de acesso granulares
- Restringir acessos
- Permitir que determinados microserviços possam se comunicar entre si



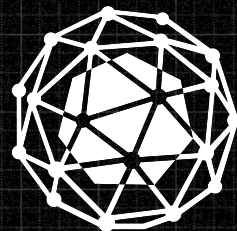
Criptografia de Tráfego e mTLS

- Protocolos de criptografia em ambas as pontas das conexões
- Toda a comunicação entre os serviços seja criptografada diretamente em trânsito
- Valida a identidade da origem e do destino antes que a conexão de fato ocorra
- Sem exigir configuração manual de certificados ou alterações nas chamadas no nível de código

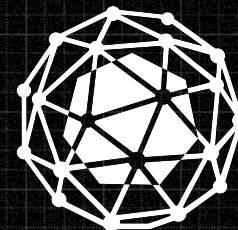
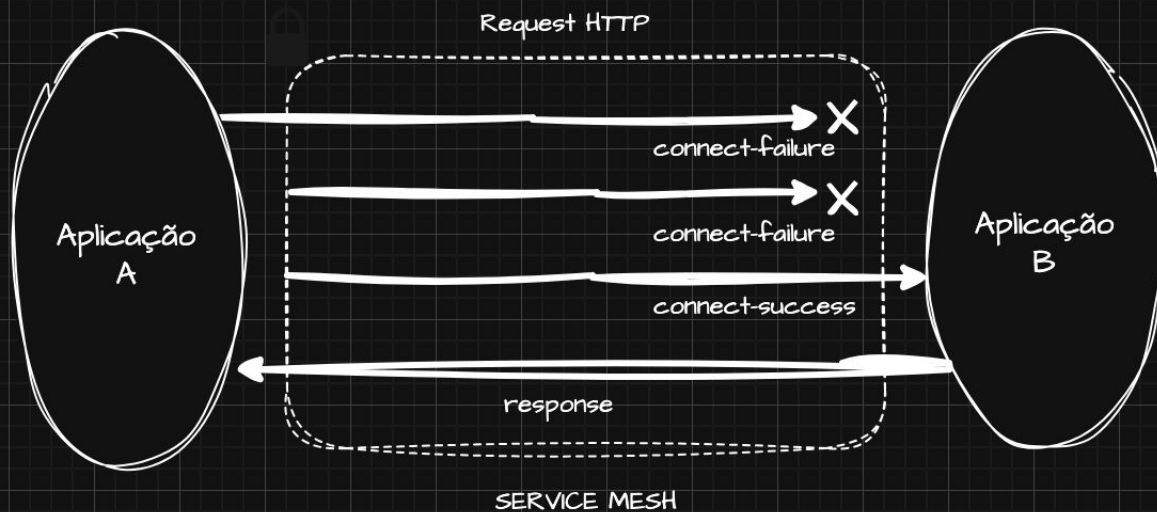


Resiliência na Camada de Comunicação

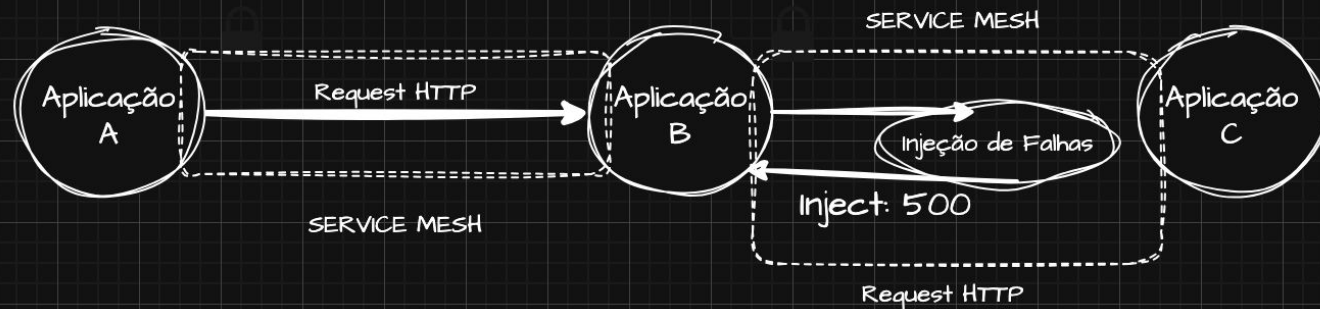
- Retries
- Connection Pool
- Circuit Breaker
- Fallbacks
- Fault Injections



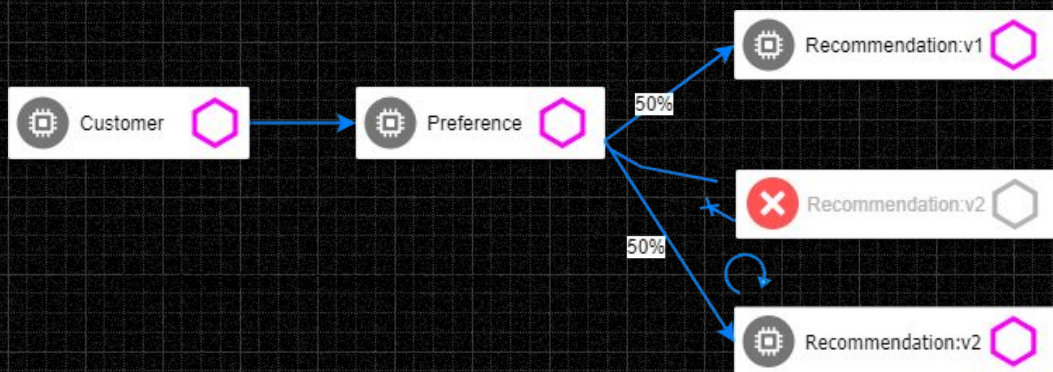
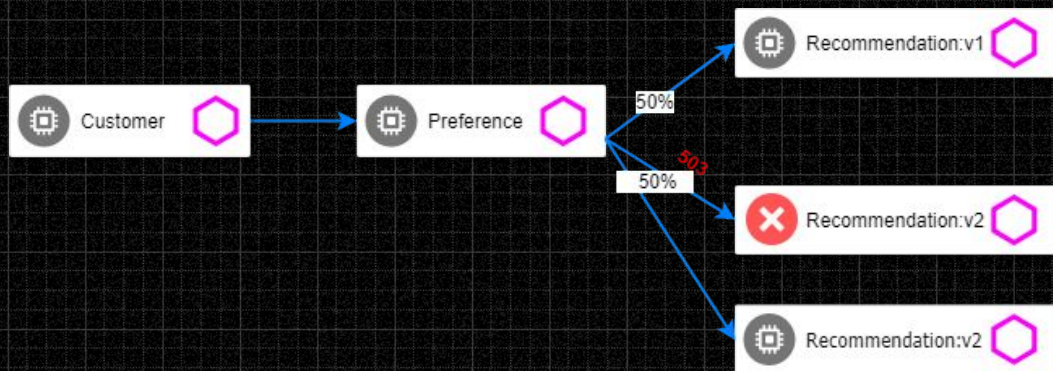
Resiliência na Camada de Comunicação

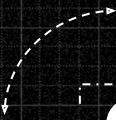


Resiliência na Camada de Comunicação



Resiliência na Camada de Comunicação



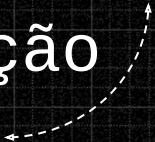


3

CONTROL PLANE E DATA PLANE

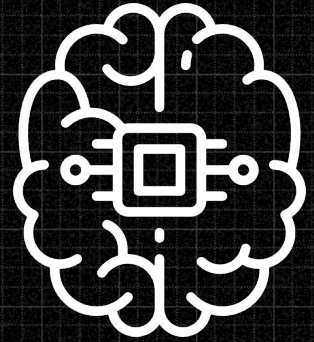


Modelos de Uso e
Operação



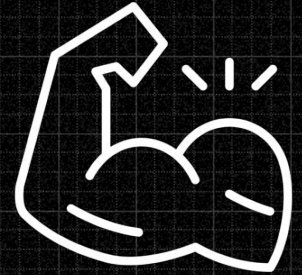
Control Plane

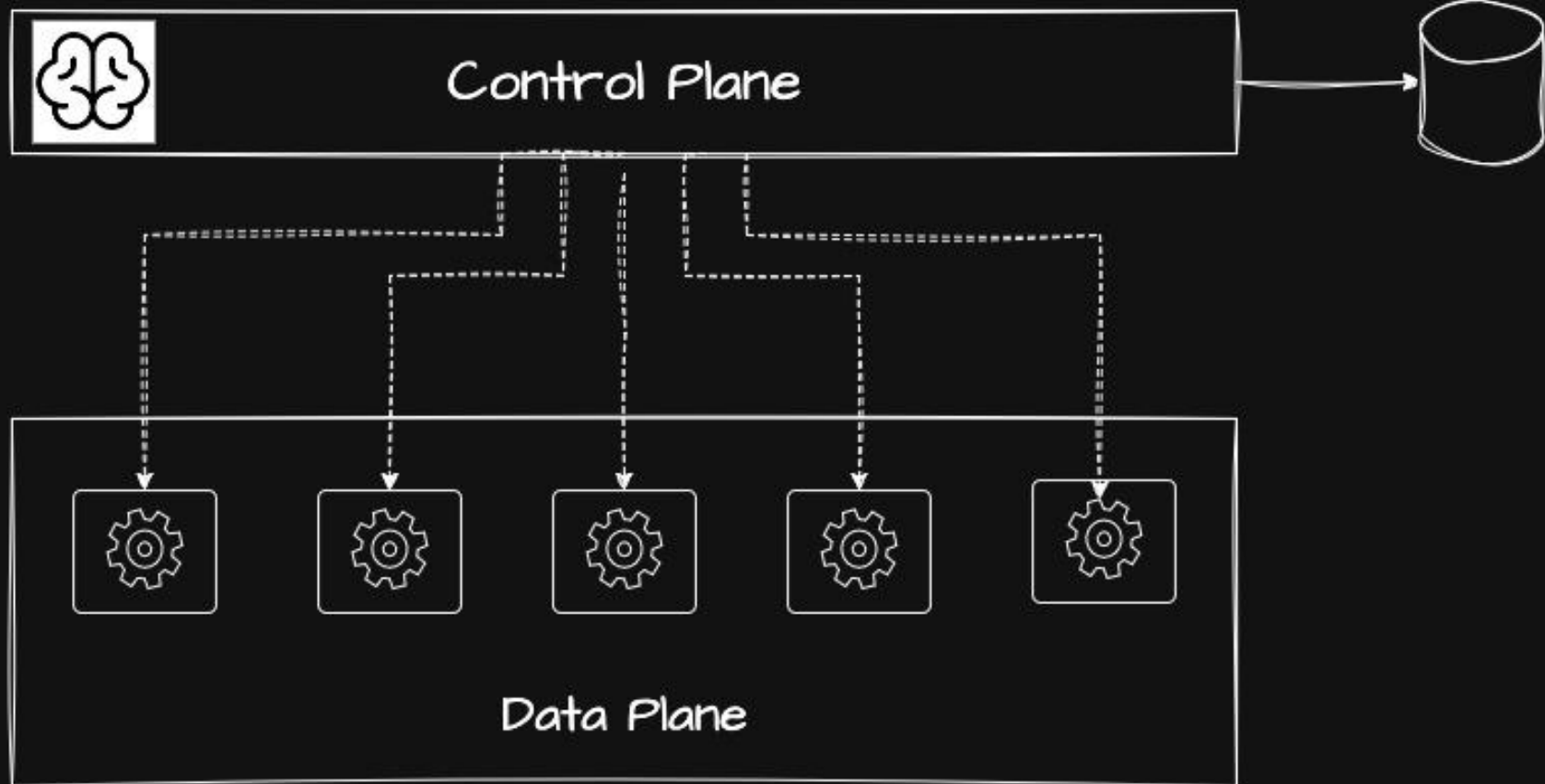
- Cérebro do Mesh
- Controlador e Planejador
- Centralização de Configuração
- Distribuição de Configuração
- Coordena o comportamento dos agentes do Dataplane
- Define o que e como será executado
- Planejador de Vão



Data Plane

- Braço do Mesh
- Executor do plano de vôo
- Coloca em prática as políticas definidas no Control Plane
- Proxies, Sidecars, CNI's e etc
- Funcionamento
 - Intercepta
 - Roteia
 - Manipula







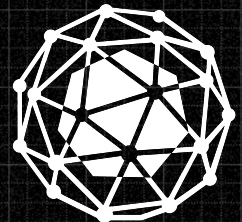
4

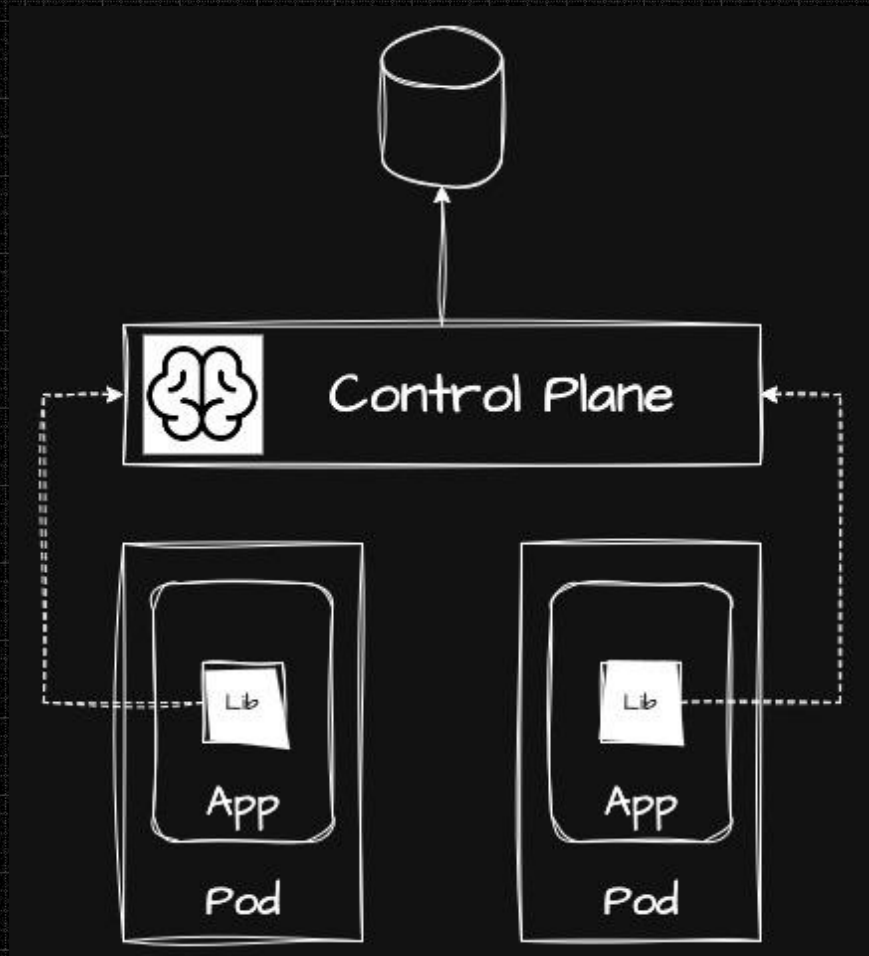
Modelos de Service Mesh

Implementações

CLIENT / SERVER (SDK)

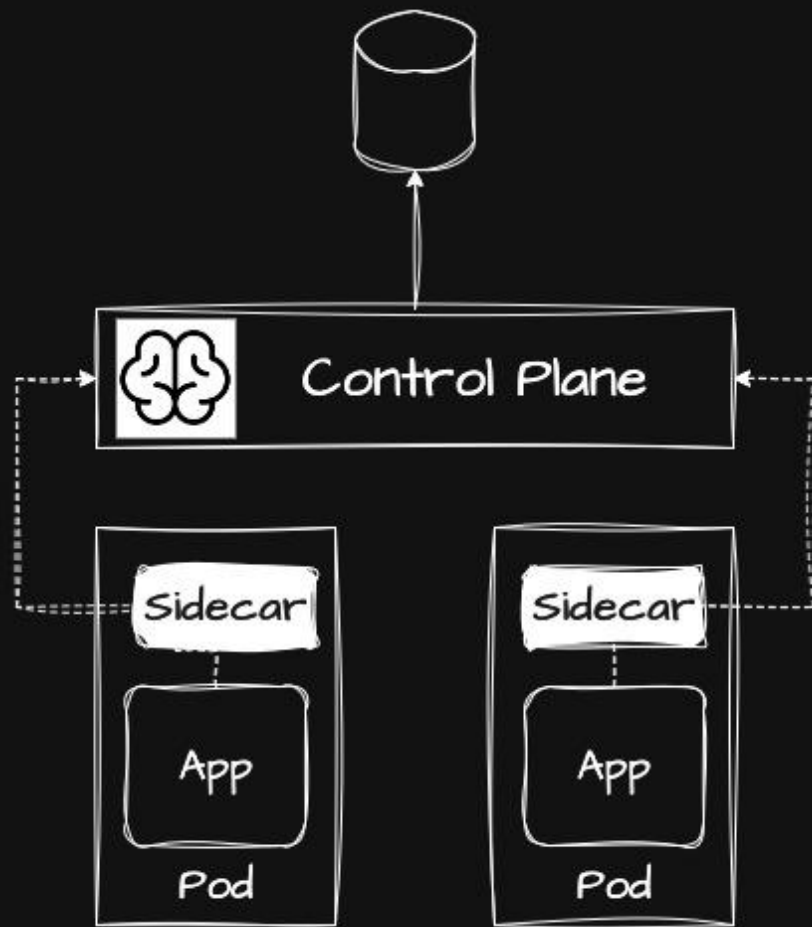
- Implementação de SDK's
- Baseado em Implementações Pragmáticas
- Consultas Periódicas no Control Plane
- Service Discovery
- Control Plane como Ponto de Falha
- Linguagens e SDKs
 - Golang
 - Java
 - .NET
 - Javascript





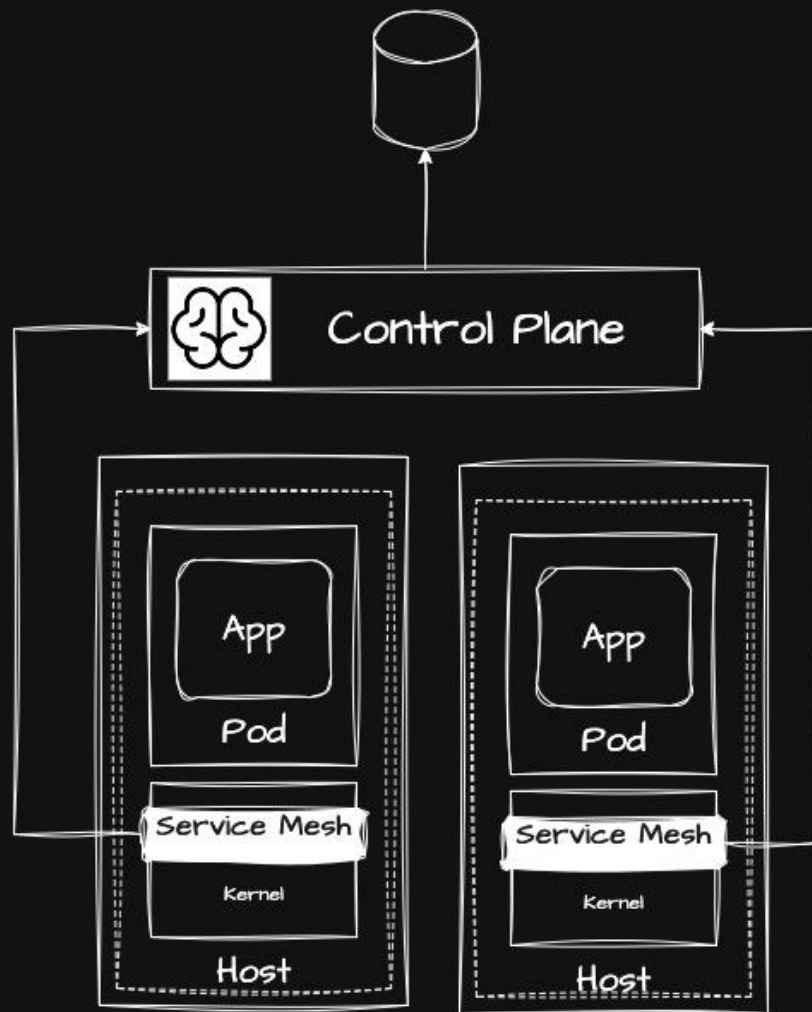
PROXY E SIDECARS

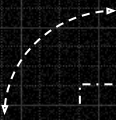
- Container Adicional
- Interceptação de Tráfego
- Layer 4 e Layer 7
- Criptografia mTLS
- Headers, Params, Paths, URL's
- Traffic Split
- Reescrita de Dados
- Compensação
- Retentativas, Circuit Breakers



Sidecarless / Proxyless

- Sem a necessidade de componentes intermediários no Data Plane
- Interceptações feitas no Kernel
- Injetar funções direto nas SysCalls
 - Metrificar
 - Rotear
 - Autenticar
 - Funções Segurança
- ZTunnel (Zero Trust Tunnel)
- eBPF



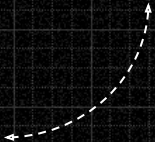


5

eBPF



extended Berkeley
Packet Filter



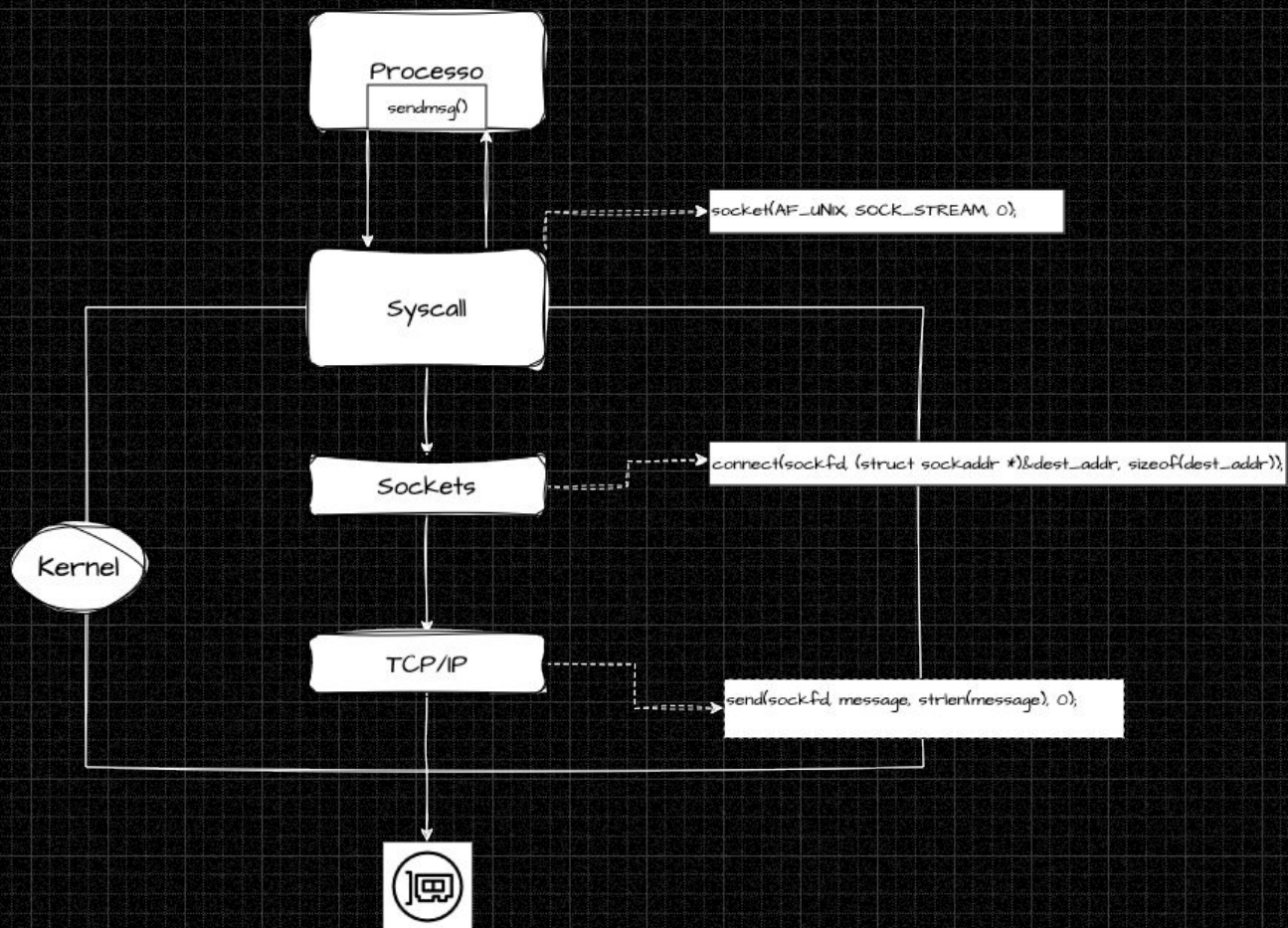
eBPF (extended Berkeley Packet Filter)

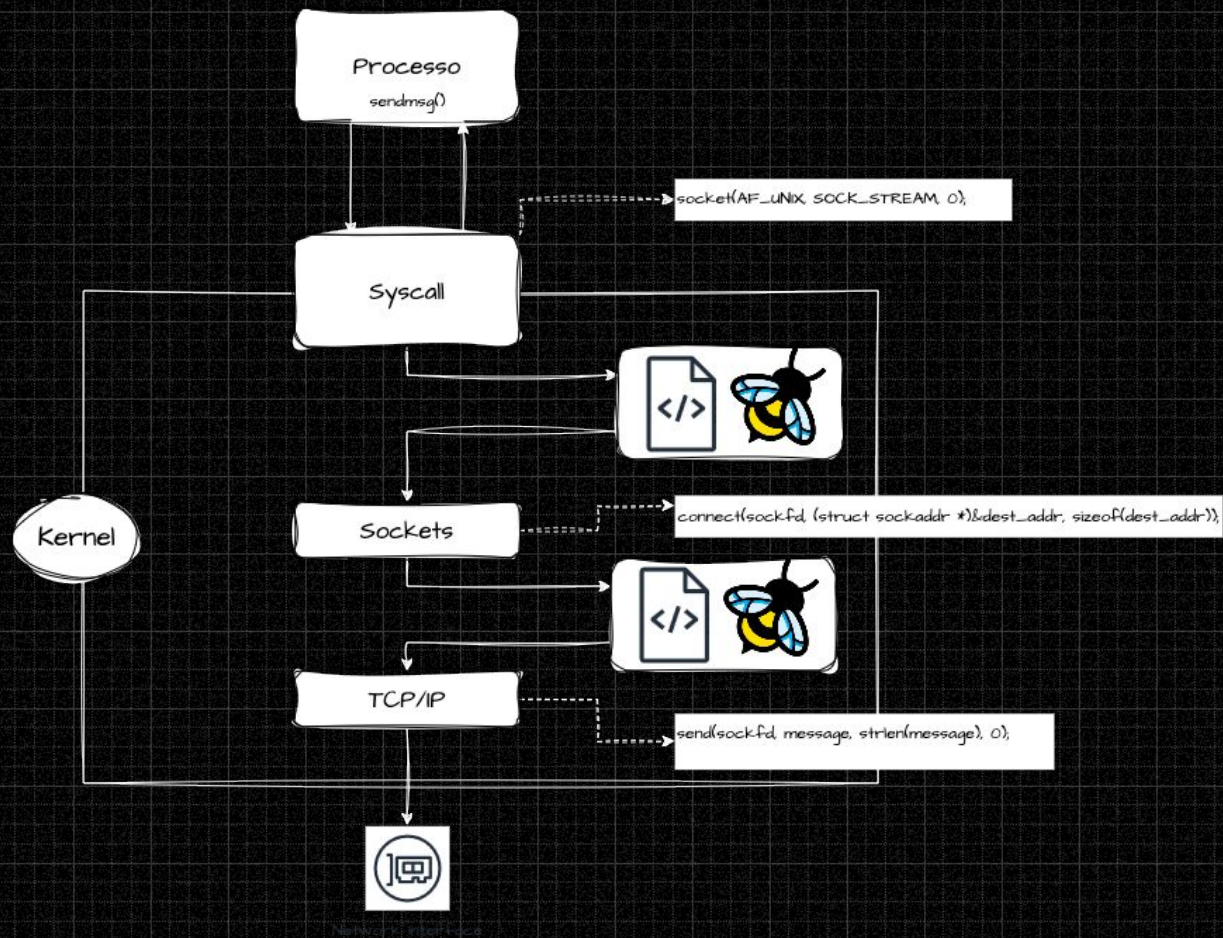
- Evolução do BPF (Berkeley Packet Filter)
- Execução de Programas Externos no Kernel
- Sem necessidade de Compilação
- Bytecode injetado no Kernel (VM)
- JIT Compiler (Just in Time)
- BCC e Verificador eBPF
- Programas KProbes (Kernel Space)
- Programas UProbes (User Space)





INTERCEPTAR E INJETAR CÓDIGO ENTRE SYSCALLS





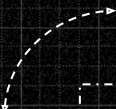
%eax	Name	Source	%ebx	%ecx	%edx	%esx	%edi	Description
1	sys_exit	kernel/exit.c	int	-	-	-	-	terminate the current process
2	sys_fork	arch/i386/kernel/process.c	struct pt_regs	-	-	-	-	create a child process
3	sys_read	fs/read_write.c	unsigned int	char *	size_t	-	-	read from a file descriptor
4	sys_write	fs/read_write.c	unsigned int	const char *	size_t	-	-	write to a file descriptor
5	sys_open	fs/open.c	const char *	int	int	-	-	open a file or device
6	sys_close	fs/open.c	unsigned int	-	-	-	-	close a file descriptor
7	sys_waitpid	kernel/exit.c	pid_t	unsigned int *	int	-	-	wait for process termination
8	sys_creat	fs/open.c	const char *	int	-	-	-	create a file or device ("man 2 open" for information)
9	sys_link	fs/namei.c	const char *	const char *	-	-	-	make a new name for a file
10	sys_unlink	fs/namei.c	const char *	-	-	-	-	delete a name and possibly the file it refers to
11	sys_execve	arch/i386/kernel/process.c	struct pt_regs	-	-	-	-	execute program
12	sys_chdir	fs/open.c	const char *	-	-	-	-	change working directory
13	sys_time	kernel/time.c	int *	-	-	-	-	get time in seconds
14	sys_mknod	fs/namei.c	const char *	int	dev_t	-	-	create a special or ordinary file
15	sys_chmod	fs/open.c	const char *	mode_t	-	-	-	change permissions of a file
16	sys_lchown	fs/open.c	const char *	uid_t	gid_t	-	-	change ownership of a file
18	sys_stat	fs/stat.c	char *	struct _old_kernel_stat *	-	-	-	get file status
19	sys_lseek	fs/read_write.c	unsigned int	off_t	unsigned int	-	-	reposition read/write file offset
20	sys_getpid	kernel/sched.c	-	-	-	-	-	get process identification
21	sys_mount	fs/super.c	char *	char *	char *	-	-	mount filesystems
22	sys_oldumount	fs/super.c	char *	-	-	-	-	unmount filesystems
23	sys_setuid	kernel/sys.c	uid_t	-	-	-	-	set real user ID
24	sys_getuid	kernel/sched.c	-	-	-	-	-	get real user ID
25	sys_stime	kernel/time.c	int *	-	-	-	-	set system time and date
26	sys_ptrace	arch/i386/kernel/ptrace.c	long	long	long	long	-	allows a parent process to control the execution of a child process
27	sys_alarm	kernel/sched.c	unsigned int	-	-	-	-	set an alarm clock for delivery of a signal

```

6  # Lista de IP's proibidos
7  deny_list = ["14.214.125.8", "55.214.20.8"]
8
9  # Conversão da lista em binário
10 deny_list_bin = [socket.inet_aton(ip) for ip in deny_list]
11
12 # Programa BPF
13 ▾ bpf_program = """
14 #include <uapi/linux/ptrace.h>
15 #include <linux/sched.h>
16 #include <net/sock.h>
17 #include <bcc/proto.h>
18
19 BPF_HASH(deny_list, u32);
20
21 int trace_connect(struct pt_regs *ctx, int sockfd, struct sockaddr *addr, int addrlen) {
22     struct sockaddr_in *addr_in = (struct sockaddr_in *)addr;
23
24     if (addr->sa_family == AF_INET) {
25         // Recupera o IP do que será realizada a conexão
26         u32 ip = addr_in->sin_addr.s_addr;
27
28         // Verifica se o IP está na deny list
29         if (deny_list.lookup(&ip)) {
30             bpf_override_return(ctx, -1); // Bloqueia a conexão retornando -1
31             return -1;
32         }
33     }
34
35     return 0; // Libera a conexão
36 }
37 """

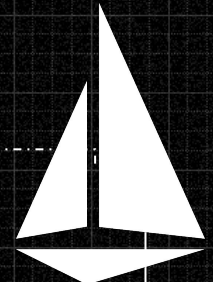
```



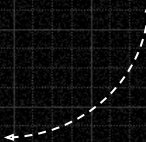


6

zTunnel (Ambient Mode)

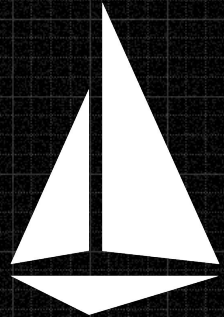


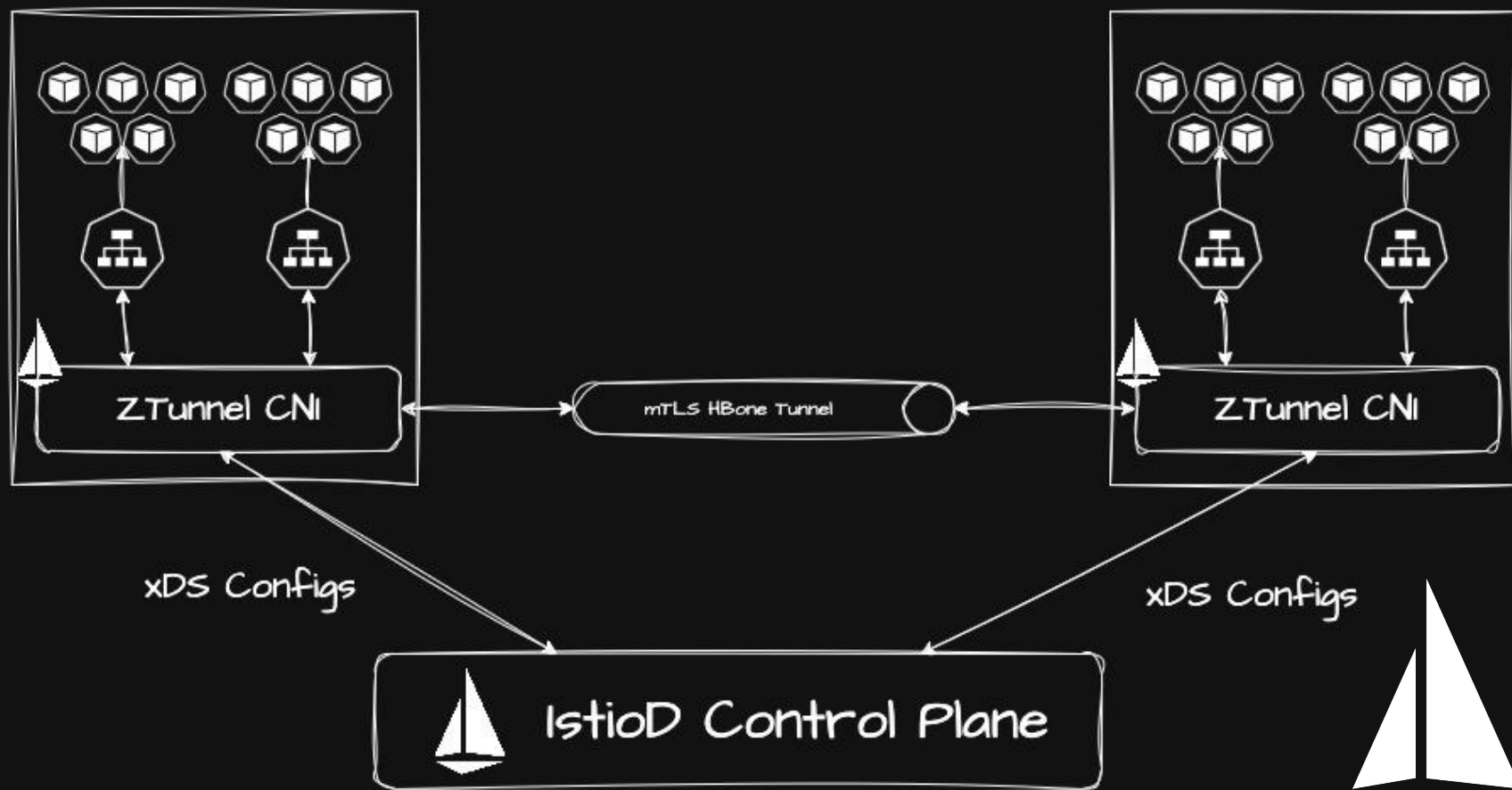
Zero Trust Tunnel
Istio

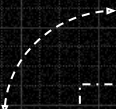


Ambient Mesh - zTunnel

- Zero Trust Tunnel
- Eliminar Overhead de Sidecars
- Layer 4 Only
- Interceptação por IPTables
- Suporte por Envoy por Namespace
- Daemonset (CNI)
- HBone (HTTP-Based Overlay Network Environment)

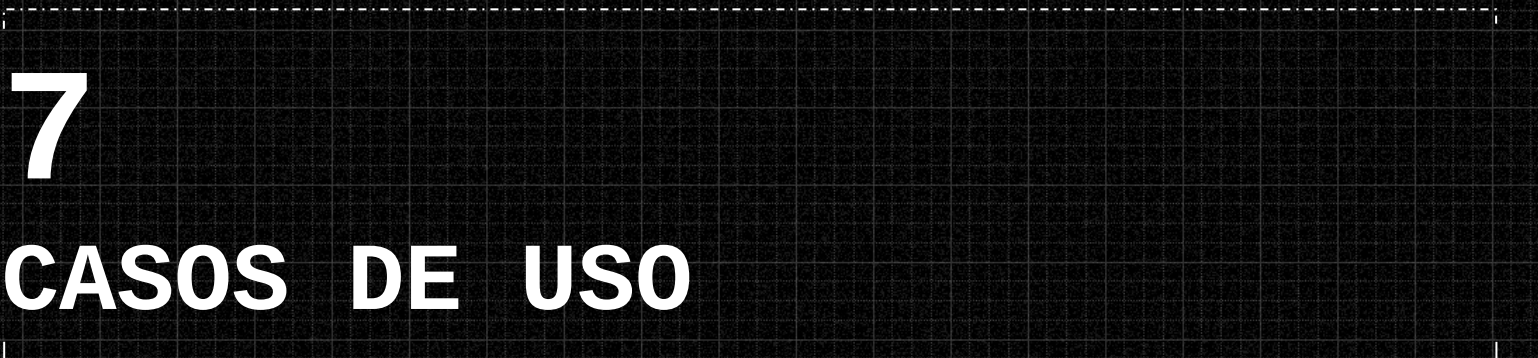




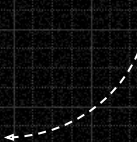


7

CASOS DE USO

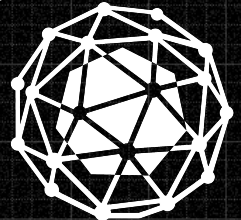


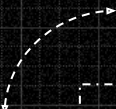
Onde e Quando



Casos de Uso

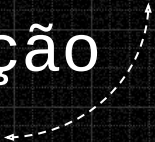
- Ambiente Granular
- Muitos Microserviços
- API Composition Pattern
- Alta Comunicação Síncrona
- Segurança de Dia-0
- Controle de Implantação
- Resiliência na Camada de Comunicação
- Multi-Cluster (Ativo/Ativo)





8

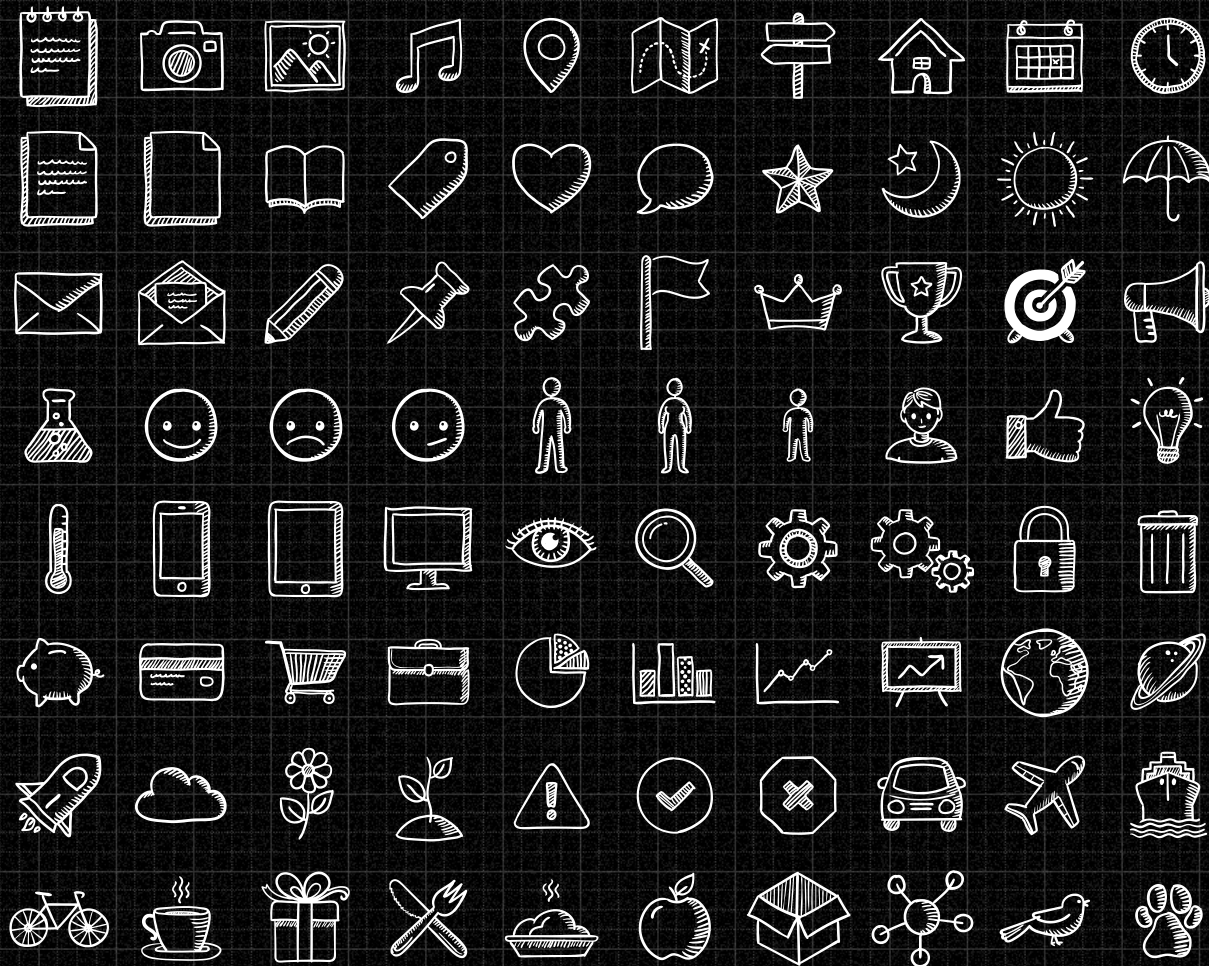
COMPARATIVOS DE MERCADO



Modelos de Uso e
Operação



	Istio Sidecars	Istio AM	Linkerd	Cilium eBPF
Arquitetura	Sidecar (Envoy)	Proxyless (Hbone + ZTunnel)	Sidecar (linkerd-proxy)	Kernel Based
Overhead	Alto	Média	Alta	Baixa
Segurança	Auth + mTLS	Auth + mTLS	Auth + mTLS	Auth + mTLS
Obsevability	Layer 7	Layer 4 + 7	Layer 7	Layer 4 + 7
Resiliência (Circuit Breakers, Retries, Load Balancing)	Sim, Completo	Sim, Completo	Menos personalizável	Limitado
Roteamento Avançado (Traffic Shifting, Canary, Blue/Green)	Sim, Completo	Sim, Completo	Sim, Completo	Limitado



SlidesCarnival icons are editable shapes.

This means that you can:

- Resize them without losing quality.
- Change fill color and opacity.

Isn't that nice? :)

Examples:

